



Guion de Sergio — Mensajería + Pago + Promos + Cierre

Tu parte de la presentación de Krypton. Hablás en **4 slides**, en **dos momentos**: tus tres servicios (slides 17–19) y el cierre final de toda la charla (slide 24).

Cómo usar esto: el texto entre «comillas» es para LEER en voz alta. Mejor todavía: aprendételo y decilo natural. Lo de "En pantalla" es lo que se ve en la diapo (pocas palabras); el detalle lo ponés vos al hablar.

Consejo: vos cerrás la presentación. El slide 24 marca el tono final — terminá con energía y abrí las preguntas con una sonrisa.

Tus turnos: Slides **17, 18, 19** (RabbitMQ, pago, promos) y **24** (cierre). **Duración:** ~3–4 min.

Slide 17 — RabbitMQ: mensajería asíncrona

En pantalla: Síncrono (Feign) = llamada telefónica · Asíncrono (RabbitMQ) = WhatsApp · order publica "se creó una orden" → notification lo consume por su cuenta · desacople total.

Decís:

«Gracias, Cristófer. Hasta ahora vimos que los servicios se hablan con Feign, que es síncrono. Yo les traigo la otra forma: la mensajería asíncrona, con RabbitMQ. La diferencia es como una llamada telefónica versus un WhatsApp. Con Feign, llamas y te quedás esperando la respuesta. Con RabbitMQ, mandás el mensaje y seguís con lo tuyo. En nuestro caso, cuando se crea una orden, el servicio de órdenes publica un mensaje que dice "se creó una orden", y sigue. El servicio de notificaciones lo consume por su cuenta, cuando puede. Order ni se entera de quién escucha. Y la ventaja: si mañana quiero sumar otro que escuche ese evento, no toco order para nada.»

Slide 18 — payment-service

En pantalla: Pago de una orden (`POST /api/orders/{id}/pay`) · order delega el cobro a **payment por Feign** · máquina de estados: PENDIENTE → CONFIRMADA · 402 si se rechaza · 422 si la transición no es válida.

Decís:

«Después tenemos el servicio de pagos. Cuando se paga una orden, con un POST a `/api/orders/{id}/pay`, el servicio de órdenes le delega el cobro al payment-service por Feign. Y acá usamos una máquina de estados: la orden pasa de PENDIENTE a CONFIRMADA solo si el pago se aprueba. Si se rechaza, devolvemos un error 402. Y si alguien intenta una transición que no corresponde —por ejemplo, pagar dos veces— devolvemos un 422. La máquina de estados nos protege de eso.»

Slide 19 — promo-service

En pantalla: Cupones · tipo PORCENTAJE o MONTO · admin crea/lista · `POST /api/promos/apply` calcula el descuento · base `krypton_promos` .

Decís:

«Y el último servicio es el de promociones, los cupones. Un cupón puede ser por porcentaje o por monto fijo. El administrador los crea y los lista, y cuando alguien aplica un cupón en el checkout, este servicio calcula el descuento.

Por ejemplo, un 10% sobre 2799 soles son casi 280 soles menos. Tiene su base propia, krypton_promos. Y este es el servicio que mencionaba Cristofer cuando hablaba del cupón en el checkout.»

👉 **Traspaso:** «Con todos los servicios cubiertos, le paso a Jason para el tramo final.»

🔴 Pausa — ahora hablan otros

Acá hablan **Jason** (20–21) y **Cristofer** (22–23, la demo y las conclusiones). **Tu próximo turno es el Slide 24, el cierre.** Cristofer cierra el slide 23 diciendo "*le dejo la palabra a Sergio*" — esa es tu entrada.

Slide 24 — Cierre + preguntas

En pantalla: "¡Gracias!" · repo en GitHub · "¿Preguntas?".

Decís:

«Y con esto cerramos. Tomamos un monolito que funcionaba y lo convertimos en nueve microservicios independientes, aplicando todos los patrones que vimos hoy. Todo el código y las guías están en el repositorio de GitHub. Les agradecemos un montón la atención... y ahora sí, ¿alguna pregunta?»

✅ Vos cerrás el show. Terminá tranquilo y dejá la puerta abierta a las preguntas.